

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 14. Functions

Lecture: 164. What are Modules ?

Section 1: Lecture Summary

This lecture introduces the concept of modules in Python, explaining that modules are collections of predefined variables, functions, classes, and objects grouped together for reuse. They are analogous to libraries or packages in other programming languages. Python comes with many built-in modules for tasks like math computations, date and time handling, and regular expressions. The lecturer demonstrates different ways to import these modules and use their contents. Furthermore, the lecture covers how to create your own modules by writing Python programs containing variables and functions, then importing these programs as modules in other Python scripts. The importance of the special `__name__` variable is explained to control whether code inside the module runs only when the module is executed directly, avoiding unwanted code execution when the module is imported.

Section 2: Key Concepts and Explanations

- What Are Modules?

- Modules (called libraries or packages in other languages) are collections of related variables, functions, classes, and objects.
- Python has many built-in modules such as:
 - ``math`` module: mathematical functions.
 - ``datetime`` module: date and time handling.
 - ``re`` module: regular expressions.
- Using modules avoids rewriting code and promotes code reuse.

- Importing Modules

- Basic import:

```
import re
```

Use module contents with prefix, e.g., ``re.search()``.

- Import with alias:

```
import datetime as dt
```

Access using shorter alias, e.g., `dt.datetime.now()`.

- Import all () contents:

```
from math import *
```

Use functions directly without prefix, e.g., `sqrt(4)`.

- Writing Your Own Module

- Any Python program (.py file) can serve as a module.

- Write variables and functions inside a .py file, e.g., `myModule.py`.

- Import your module in another script:

```
import myModule as m
print(m.data)
print(m.add(5, 6))
print(m.sub(8, 5))
```

- This allows reuse of your code across multiple programs.

- Controlling Code Execution in Modules

- When running a Python file directly, the built-in variable `\_\_name\_\_` equals ` "\_\_main\_\_" `.

- When importing the file as a module, `\_\_name\_\_` equals the module's filename.

- Use the following conditional in your module to ensure some code runs only when the file is executed directly:

code to run only when executed directly

```
if __name__ == "__main__":
    print(add(10, 5))
    print(sub(10, 5))
```

- This prevents print statements or test code in the module from running when imported elsewhere.

Section 3: Example Code and Use Cases

Example module (`myModule.py`):

```
data = 500
```

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

if __name__ == "__main__":
    print("Sum:", add(10, 5))
    print("Difference:", sub(10, 5))
```

- This module defines a variable `data` and two functions `add` and `sub`.
- The print statements run only if `myModule.py` is executed directly.

Example program using the module (.myProgram.py):

```
import myModule as m

print("Data:", m.data)
print("Add:", m.add(5, 6))
print("Sub:", m.sub(8, 5))
```

- Imports `myModule` with alias `m`.
- Accesses variable `data` and calls functions `add` and `sub` via the alias.

Section 4: Key Takeaways

- Python modules group related reusable code: variables, functions, classes, and objects.
- Modules can be imported using `import module`, `import module as alias`, or `from module import`.
- Any Python script file (.py) can be used as a module by importing it into another script.
- Use the `if \_\_name\_\_ == "\_\_main\_\_":` guard in your module to prevent execution of test or demo code upon import.
- Writing modular code improves code reuse and organization in Python projects.